Software Department Task

Ideal Braking Curve

Queen Mary Formula Student Team
By Samaira Khan

Table of Contents

1. Purpose of This Report 3
2. Why I Chose Python 3
3. Background and Equations 3
  3.1. What the Curve Shows 3
  3.2. The Equations 3
  3.3. What the Equations show 4
4. Decisions and Assumptions 4
  4.1. Range of μ 4
  4.2. Showing μ on the Graph 5
  4.3. Assumptions 5
5. How the Code Works 5
  5.1. Setting Up 5
  5.2. The Three Functions 6
  5.3. Calculating, Checking and Printing 6
  5.4. Drawing the Graph 7
  5.5. Saving the Graph 7
6. Results 8
7. Use of LLM's 8
  7.1. What I used LLM's for 8
  7.2. Changes to the Graph 9
8. References 10

1. Purpose of This Report

This report explains the Python program I wrote for the software department. The program works out the ideal braking curve for our Formula Student car and draws it as a graph. It also prints the numbers in a table so they can be checked or reused.

The report covers why I picked Python, the equations I used, the decisions and assumptions I made, how the code works, and what the results show. The steps for running the program are in the README file in the project folder.

2. Why I Chose Python

I chose Python and its libraries for the following reasons:

- It is free and runs on Windows, Mac and Linux, so everyone in the team can install the same thing.
- It is easy to read. The lines look like the maths, so a team member who knows the theory can follow the code without learning much syntax.
- NumPy lets me do a calculation on a whole list of friction coefficients at once, so I do not need to write loops.
- Matplotlib draws graphs with labelled axes and saves them as image files, which is what the task asks for.
- There is nothing to compile. You write the file and run it.

I also have some Java experience, but a simple graph needs more set-up in Java, so Python was the easier choice for a task this size.

3. Background and Equations

3.1. What the Curve Shows

The ideal braking curve plots the rear braking force against the front braking force at the point where both pairs of tyres are just about to slip. Brake engineers draw it before they design any brake parts, because it only needs the car's dimensions and the grip of the tyres *(The Autopian, n.d.)*.

3.2. The Equations

The task gives the car values in Table 1 (these were given in the task sheet). They are written once at the top of the program as well.

Quantity	Symbol	Value	Unit
Acceleration due to gravity	g	9.81	m/s^2
Mass of the car	m	280	kg
Height of centre of gravity	h_cg	0.25	m
Wheelbase	d_wheel	1.590	m
Weight share on the front axle	Ψ_front	0.5	no unit

Table 1: Car values used in the program

The (longitudinal) weight that moves from the rear axle to the front axle while braking is:

$$\Delta W = a \cdot m \cdot h_cg \,/\, d_wheel$$

The braking force each axle can give before its tyres slip is the friction coefficient (μ) multiplied by the load on that axle. The formulas for the force on the front and rear axles as given in the task sheet are:

$$F_front = \mu\,(m\,g\,\Psi_front + \Delta W)$$

$$F_rear = \mu\,(m\,g\,(1 - \Psi_front) - \Delta W)$$

3.3. What the Equations show

The weight transfer needs the acceleration a, but the task only gives μ. So I first worked through one value of μ by hand to see how the two are linked. Adding the front and rear equations makes the ΔW terms cancel to give:

$$F_front + F_rear = \mu\, m\, g$$

From Newton's second law:

$$a = \mu\, g$$

This means every μ gives one acceleration, one weight transfer, and one pair of front and rear forces. That pair is one point on the graph.

The equation F_front + F_rear = μ m g is also a straight line on the graph.

4. Decisions and Assumptions

4.1. Range of μ

I used μ from 0 to 2.0 in steps of 0.1, which gives 21 points.

- **Why up to 2.0:** GripCalc lists dry tarmac values of about 0.70 - 0.85 for economy road tyres, 0.90 to 1.10 for performance tyres, 1.10 to 1.40 for track-

day tyres and 1.50 to 1.80 for racing slicks (*GripCalc, n.d.*). HPWizard also says racing tyres can reach about 1.8 (*HPWizard, n.d.*). Formula Student cars use slicks, so μ above 1 is realistic up until 2.0 as it has some leeway past the highest value.

- **Why start at 0:** with no grip there is no braking force, so the curve starts at the origin, like the curve in the video.
- **Why steps of 0.1:** it gives enough points for a smooth curve and the values are easy to read.

4.2. Showing μ on the Graph

I drew one diagonal line for each μ and wrote the value where the line meets the x-axis, as in the video. Reading the graph is then simple: find the μ you want, follow its line up to the curve, and read the front and rear forces from the axes.

4.3. Assumptions

These are the assumptions I made and their reasons:

- The car brakes in a straight line on flat ground, because the equations have no terms for corners or slopes.
- There is no aerodynamic downforce or drag, because they are not in the equations.
- The same constant μ is used at the front and rear.
- There is no rolling resistance (or it is very small compared to the braking forces.)
- The brakes are perfect, so the car decelerates at exactly μ g (the curve I ideal.)
- All values are in SI units.

5. How the Code Works

All of the code is in one file, `main.py`.

5.1. Setting Up

The program starts by importing three things. `numpy` does the calculations, `matplotlib.pyplot` draws the graph, and `os` is used later to save the file.

```
import os
import numpy as np
import matplotlib.pyplot as plt
```

Next are the car values from Table 1. I used the same names as the symbols in the task, and each value has its unit in a comment.

```
g = 9.81          # m/s^2
m = 280           # kg
h_cg = 0.25       # m
d_wheel = 1.590   # m
psi_front = 0.5
```

`np.linspace(0, 2, 21)` makes a list of 21 numbers from 0 to 2 with equal gaps, which are the friction coefficients.

```
mu = np.linspace(0, 2, 21)
```

5.2. The Three Functions

There is one function for each equation. `weight_shift` first finds the acceleration with a = μ g, then calculates ΔW.

```
def weight_shift(mu):
    a = mu * g
    return a * m * h_cg / d_wheel
```

`front_force` and `rear_force` then use the weight shift in the two force equations.

```
def front_force(mu):
    return mu * (m * g * psi_front + weight_shift(mu))


def rear_force(mu):
    return mu * (m * g * (1 - psi_front) - weight_shift(mu))
```

(`mu` is a NumPy list, each function does its sum for every value at once, so no loop is needed)

5.3. Calculating, Checking and Printing

The functions are called once with the whole list of μ values.

```
front = front_force(mu)
rear = rear_force(mu)
```

The `assert` line is a safety check. Section 3.3 showed that the front and rear forces must add up to μ m g. If they do not, the program stops with an error, so a mistake in the equations is noticed straight away.

```
assert np.allclose(front + rear, mu * m * g)
```

A loop then prints one row of the results table for each μ.

```
print("mu    front (N)    rear (N)")
for i in range(len(mu)):
    print(f"{mu[i]:.1f}   {front[i]:9.2f}   {rear[i]:9.2f}")
```

5.4. Drawing the Graph

The curve is drawn with the front force on the x-axis and the rear force on the y-axis.

```
plt.figure(figsize=(12, 6.5), dpi=150)
plt.plot(front, rear)
```

Then a loop draws one diagonal line for each μ, apart from μ = 0. The line goes from (0, total) to (total, 0), where total is μ m g, so front plus rear is the same along it. `ax.text` writes the μ value at the bottom of the line.

```
for k in mu[1:]:
    total = k * m * g
    ax.plot([0, total], [total, 0], color="#d0d0d0", linestyle=":")
    ax.text(total, 25, f"{k:.1f}", fontsize=7, ha="center")
```

The rest of this part sets the title, axis labels, axis limits, grid and colours so that the graph looks like the one in the video we were provided in the task sheet (more about this in Section 7.2):

```
plt.ylim(0, 1400)
plt.grid(True, color="lightgray")
plt.title("Ideal Brake Curve")
plt.xlabel("Front Brake Force (N)")
plt.ylabel("Rear Brake Force (N)")
```

5.5. Saving the Graph

The graph is saved as a PNG picture in a folder called `plots`, next to `main.py`. `os.makedirs` creates the folder if it is not there yet, so the program does not crash on a fresh download. `plt.show()` then opens the graph in a window.

```
folder = os.path.join(os.path.dirname(os.path.abspath(__file__)), "plots")
os.makedirs(folder, exist_ok=True)
plt.savefig(os.path.join(folder, "ideal_braking_curve.png"), dpi=300)
plt.show()
```

6. Results

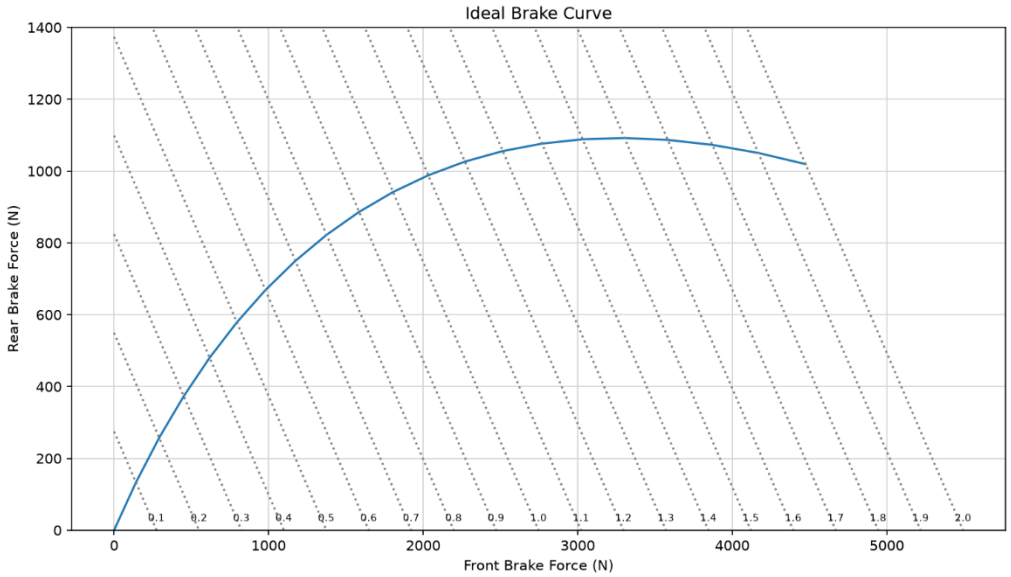


Figure 1: The ideal braking curve with a diagonal line for each friction coefficient

The curve bends because more weight moves to the front as μ increases. The front force keeps growing, but the rear force reaches its highest value of about 1092 N at μ=1.6 and then falls.

Table 2 shows some of the values printed by the program. The program prints all 21 rows.

μ	Front force (N)	Rear force (N)	Total (N)
0.2	291.96	257.40	549.36
0.5	794.67	578.73	1373.40
0.8	1375.13	822.31	2197.44
1.0	1805.29	941.51	2746.80
1.5	3031.85	1088.35	4120.20
2.0	4474.35	1019.25	5493.60

Table 2: Selected results

7. Use of LLM's

7.1. What I used LLM's for

- Research: I used it to find and summarise information about the ideal braking curve and about typical tyre friction coefficients. I specifically used values only from the sources in the References section though.
- It helped me write the README file for the project.
- Using Word: It showed me how to insert equations and code blocks into a Word doc for this report.
- Checking my hand calculations: I did some calculations by hand first and then asked it to check them, to see whether I had understood the formulae. It corrected me where I had not substituted the value of g in one of the equations.
- Formatting the graph: The task discouraged the use of LLMs for code but when I first plotted my graph, the result was not easy to read (see Figure 2) so I asked for help to make it look like the graph in the video. The details are in the section below.

7.2. Changes to the Graph

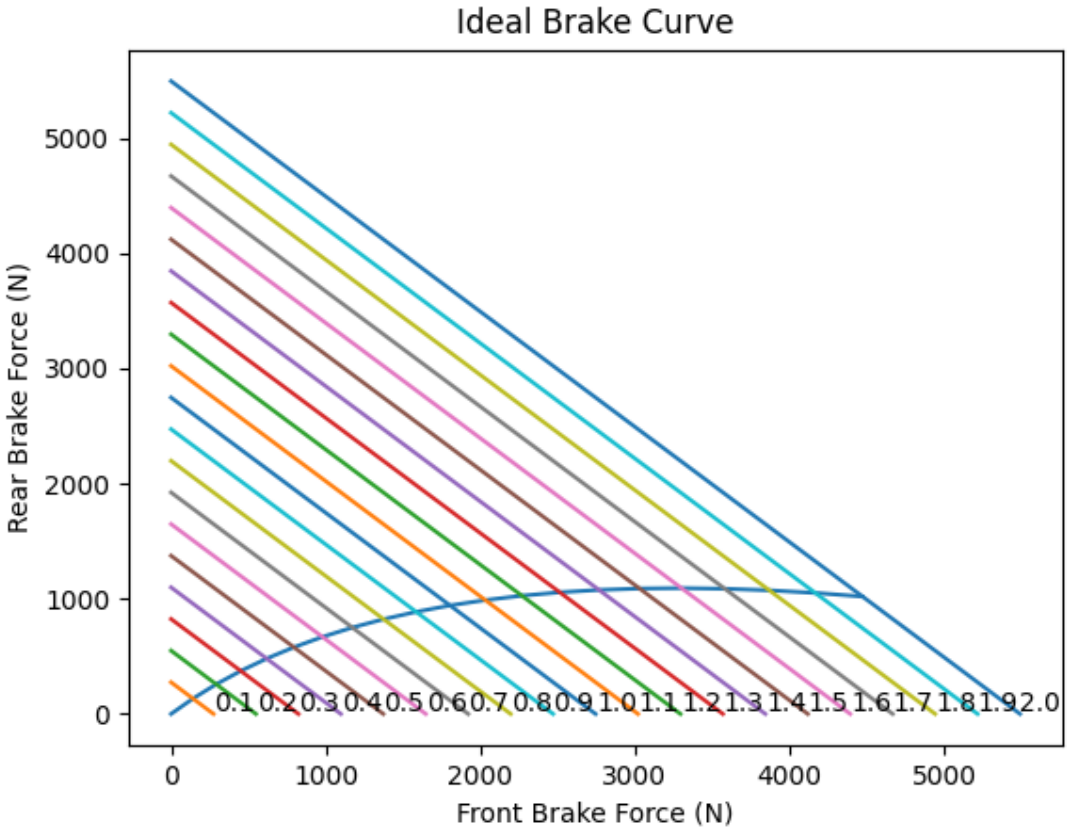


Figure 2: The initial graph before including the formatting code.

As seen from the Figure 2, first version of the graph had lines of many different colours, the friction coefficient labels were overlapping, there were no grid lines to help in reading the values and the y-axis went up to about 5500 N.

These five lines of code were added or changed to fix this (from Section 5.4.)

```
plt.ylim(0, 1400)
plt.grid(True, color="lightgray")
plt.title("Ideal Brake Curve")
plt.xlabel("Front Brake Force (N)")
plt.ylabel("Rear Brake Force (N)")
```

The rest of the code was written and debugged by me and I used the resources mentioned below to learn how to write the code to plot the graph.

8. References

GripCalc (n.d.) *Tire Grip Force Calculator – Grip, Cornering G & Braking*. Available at: https://gripcalc.com/tire-grip-calculator.html

HPWizard (n.d.) *Tire friction and rolling coefficients*. Available at: https://www.hpwizard.com/tire-friction-coefficient.html

Suspensions Explained (n.d.) *Video on the ideal braking curve*. YouTube. Available at: https://www.youtube.com/watch?v=iJ7bf20phWc

The Autopian (n.d.) *An engineer describes the ideal brake curve and why cars have such vastly different brake sizes*. Available at: https://www.theautopian.com/an-engineer-describes-the-ideal-brake-curve-and-why-cars-have-such-vastly-different-brake-sizes/

GeeksforGeeks (2025) *How to plot a smooth curve in Matplotlib?* Available at: https://www.geeksforgeeks.org/machine-learning/how-to-plot-a-smooth-curve-in-matplotlib/ (Accessed: 29 October 2026).

IIT Madras - B.S. Degree Programme (2025) *W1_L3: Numpy and matplotlib - plotting simple curves* [video]. YouTube, 12 September 2025. Available at: https://www.youtube.com/watch?v=Bc3_1dl__MA (Accessed: 29 October 2026).